

Entity Framework

Faculdade UNIDESC

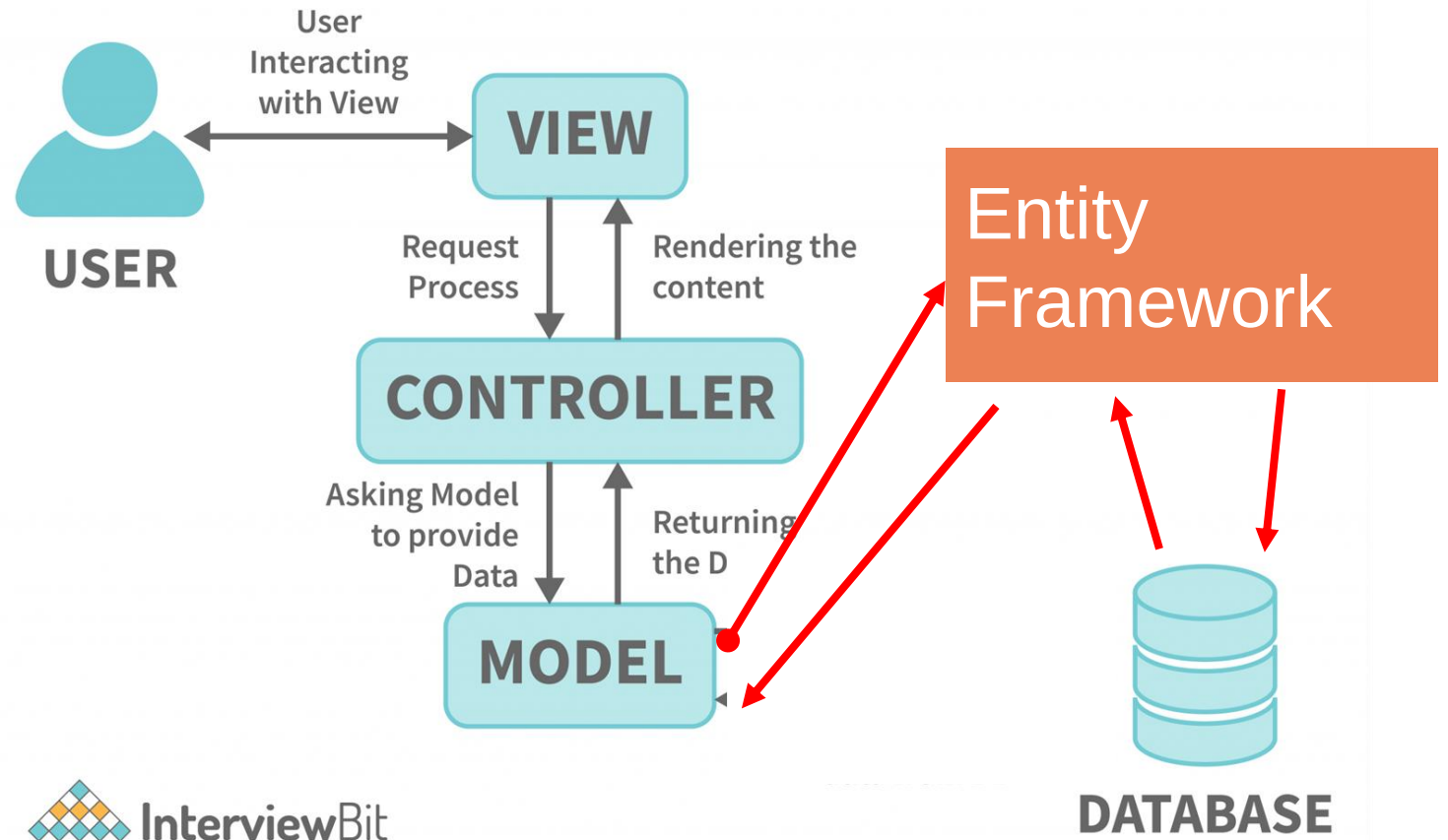
Programação Para Web II

Profº Ruben Prado

Entity Framework

- O Entity Framework é um conjunto de tecnologias no ADO.NET que dão suporte ao desenvolvimento de aplicativos de software orientados a dados. O Entity Framework permite que desenvolvedores trabalhem com dados na forma de propriedades e objetos específicos de domínio, como clientes e endereços de clientes, sem se preocupar com as tabelas e colunas do banco de dados subjacente em que os dados estão armazenados. Ele é um ORM (Object Relational Mapping).

MVC



Integração no Padrão MVC

01

Model: Responsável pelos dados e regras de negócio.

02

View: Responsável pela apresentação.

03

Controller: Coordena a interação entre o Model e a View, manipulando as requisições e definindo a resposta.

Classes

EF Core

BD

As Classes de Modelo de Domínio (Model)

O Entity Framework Core mapeia os modelos

O Banco de Dados, através das Migrações, são criados e estruturados de acordo com as configurações dos Modelos, incluindo relacionamentos



Vantagens

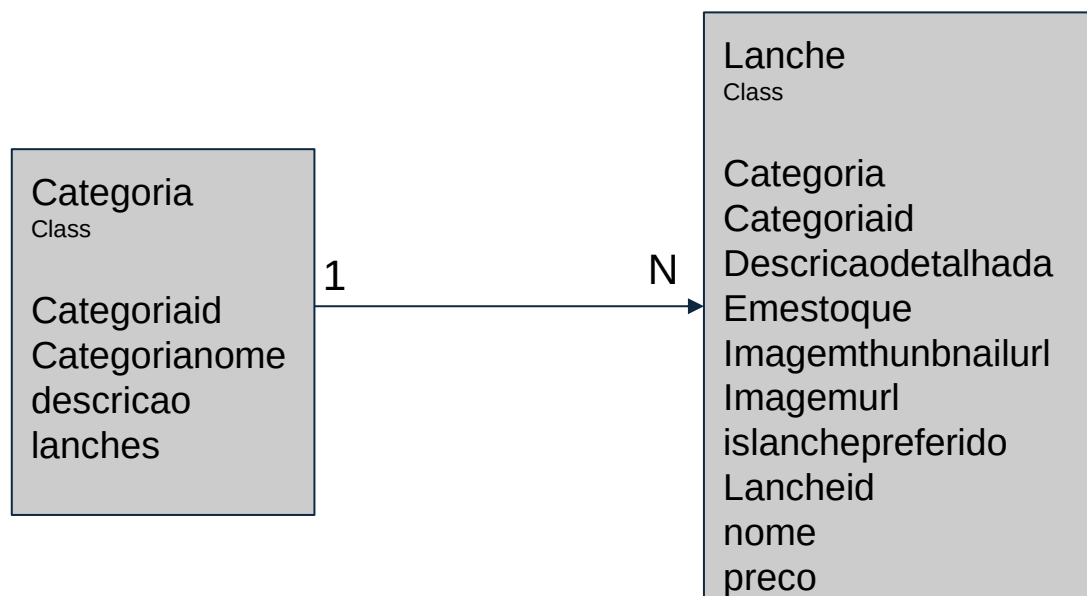
- Utilizar A linguagem C#
- Abstrair Comandos SQL
- Versionamento através das migrações (Migrations)
- Redução do tempo de desenvolvimento
- Multiplataforma e OpenSource
- Suporta diversos BD: SQL Server, MySQL, PostgreSQL, SQLite
- Suporta Abordagens: Code First e DataBase First
- Instalado via Nuget
- Suporta ferramenta de linha de comando

Convenções

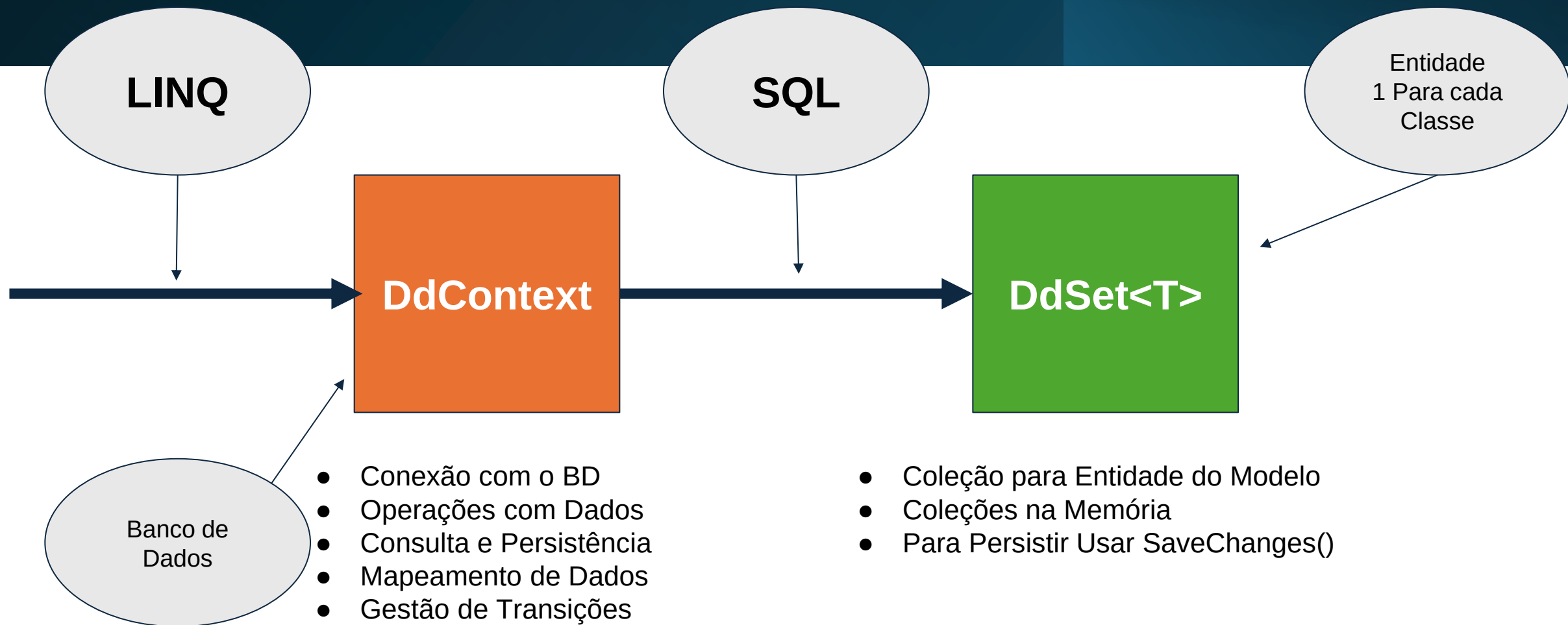
- A propriedade Id ou <nome_entidade>Id, vai gerar uma chave primária na tabela;
- As propriedades definidas na classe irão gerar colunas com o mesmo nome na tabela;
- O tipo de dados das colunas geradas a partir dos tipos definidos nas propriedades depende do servidor de banco de dados;

C#	SQL SERVER
int	int
string	nVarchar(max)
decimal	decimal(10,2)
bool	bit
dateTime	dateTime

A definição dos relacionamentos, entre as entidades são definidas em uma propriedade de navegação.



Relacionamento:
Um para muitos



Instalação e Configuração

- A propriedade Id ou <nome_entidade>Id, vai gerar uma chave primária na tabela;
- As propriedades definidas na classe irão gerar colunas com o mesmo nome na tabela;
- O tipo de dados das colunas geradas a partir dos tipos definidos nas propriedades depende do servidor de banco de dados;

Pacotes Nuget

EF



Microsoft.EntityFrameworkCore by Microsoft

Entity Framework Core is a modern object-database mapper for .NET. It supports LINQ queries, change tracking, updates, and schema migrations. EF Core works with SQL Server,...

EF



Microsoft.EntityFrameworkCore.Design by Microsoft

Shared design-time components for Entity Framework Core tools.

EF



Microsoft.EntityFrameworkCore.Sqlite.Core by Microsoft

SQLite database provider for Entity Framework Core. This package does not include a copy of the native SQLite library.

EF



Microsoft.EntityFrameworkCore.Tools by Microsoft

Entity Framework Core Tools for the NuGet Package Manager Console in Visual Studio.

.NET



Microsoft.VisualStudio.Web.CodeGeneration.Design by Microsoft

Code Generation tool for ASP.NET Core. Contains the dotnet-aspnet-codegenerator command used for generating controllers and views.



SQLitePCLRaw.bundle_e_sqlite3 by Eric Sink

This 'batteries-included' bundle brings in SQLitePCLRaw.core and the necessary stuff for certain common use cases. Call SQLitePCL.Batteries.Init(). Policy of this bundle: e_sqlite...

Criar o AppDbContext

```
using Microsoft.EntityFrameworkCore;
using AlugaCar.Models;

namespace AlugaCar.Context
{
    13 references
    public class AppDbContext : DbContext
    {
        0 references
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        {
        }

        7 references
        public DbSet<Cliente> clientes { get; set; }
        7 references
        public DbSet<Funcionario> funcionarios { get; set; }
        7 references
        public DbSet<Veiculo> veiculos { get; set; }
        0 references
        public DbSet<Locacao> locacoes { get; set; }
        7 references
        public DbSet<Modelo> modelos { get; set; }
    }
}
```

Configuração da String de Conexão no appsettings.json

```
{  
  "ConnectionStrings": {  
    "DefaultConnection": "Data Source=AlugaCar.db"  
  },  
  "Logging": {  
    "LogLevel": {  
      "Default": "Information",  
      "Microsoft.AspNetCore": "Warning"  
    }  
  },  
  "AllowedHosts": "*" }  
}
```

Registrar o Serviço de conexão com BD

```
using Alugacar.Context;  
using Microsoft.EntityFrameworkCore;  
  
var builder = WebApplication.CreateBuilder(args);  
  
//Configuração do Banco de Dados  
builder.Services.AddDbContext<AppDbContext>(options =>  
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")));  
  
// Add services to the container.  
builder.Services.AddControllersWithViews();  
  
var app = builder.Build();
```

Migrations

- No ASP.NET Core MVC, o Entity Framework Core (EF Core) oferece o recurso de Migrações para gerenciar alterações no modelo de dados e manter o esquema do banco de dados sincronizado com essas mudanças. Isso é especialmente útil em ambientes de produção, onde descartar e recriar o banco de dados não é uma opção viável devido à necessidade de preservar dados existentes.
- As migrações permitem que você evolua o esquema do banco de dados de forma incremental, aplicando alterações como adição ou remoção de tabelas, colunas e chaves estrangeiras, sem perder os dados existentes. O EF Core compara o modelo de dados atual com um instantâneo anterior para gerar scripts que atualizam o banco de dados conforme necessário.

MIGRATIONS

Adicionar uma Migração Inicial Após configurar o DbContext e as classes de modelo, você pode criar a primeira migração com o seguinte comando:

```
dotnet ef migrations add Inicial
```

Para atualizar o banco de dados com base na migração criada, execute:

```
dotnet ef database update
```

Sempre que você modificar o modelo de dados (por exemplo, adicionando uma nova propriedade a uma entidade), crie uma nova migração:

```
dotnet ef migrations add AdicionaCampoTelefone
```

Em seguida, aplique a migração com:

```
dotnet ef database update
```

Caso precise remover a migração antes de atualizar o BD. Utilize:

```
dotnet ef migration remove
```